

1. Create a 1D NumPy array of 20 random integers between 1 and 100. Find the minimum, maximum, mean, and standard deviation without using Python built-in functions — use only NumPy aggregation functions.

Ans.

```
# Import NumPy library
```

```
import numpy as np
```

```
# Create a 1D NumPy array with 20 random integers between 1 and 100
```

```
arr = np.random.randint(1, 101, 20)
```

```
# Display generated array
```

```
print("Array:")
```

```
print(arr)
```

```
# Find minimum value using NumPy
```

```
minimum = np.min(arr)
```

```
# Find maximum value using NumPy
```

```
maximum = np.max(arr)
```

```
# Find mean (average) using NumPy
```

```
mean_value = np.mean(arr)
```

```
# Find standard deviation using NumPy
```

```
std_value = np.std(arr)
```

```
# Display results
```

```
print("\nMinimum value:", minimum)
```

```
print("Maximum value:", maximum)
print("Mean value:", mean_value)
print("Standard Deviation:", std_value)
```

2. Create a 4×4 matrix using NumPy. Perform the following operations:

- Extract the diagonal elements
- Replace all even numbers with 0
- Find the index of the maximum value using `argmax()`

Ans.

```
# Import NumPy library
```

```
import numpy as np
```

```
# Create a 4×4 matrix
```

```
matrix = np.array([
    [10, 15, 20, 25],
    [30, 35, 40, 45],
    [50, 55, 60, 65],
    [70, 75, 80, 85]
])
```

```
# Display original matrix
```

```
print("Original Matrix:")
print(matrix)
```

```
# Extract diagonal elements
```

```
diagonal = np.diag(matrix)
```

```
print("\nDiagonal Elements:")
```

```
print(diagonal)
```

```
# Replace all even numbers with 0
```

```
matrix[matrix % 2 == 0] = 0
```

```
print("\nMatrix after replacing even numbers with 0:")
```

```
print(matrix)
```

```
# Find index of maximum value using argmax()
```

```
max_index = np.argmax(matrix)
```

```
print("\nIndex of maximum value:")
```

```
print(max_index)
```

3. Write a program to create two 3×3 NumPy arrays and perform element-wise addition, subtraction, and matrix multiplication. Display results with proper labels..

Ans.

```
# Import NumPy library
```

```
import numpy as np
```

```
# Create first 3×3 array
```

```
array1 = np.array([
```

```
    [1,2,3],
```

```
    [4,5,6],
```

```
[7,8,9]
])

# Create second 3x3 array
array2 = np.array([
    [9,8,7],
    [6,5,4],
    [3,2,1]
])

# Display arrays
print("First Array:")
print(array1)

print("\nSecond Array:")
print(array2)

# Element-wise addition
addition = array1 + array2

print("\nElement-wise Addition:")
print(addition)

# Element-wise subtraction
subtraction = array1 - array2

print("\nElement-wise Subtraction:")
print(subtraction)
```

```
# Matrix multiplication
multiplication = np.dot(array1,array2)

print("\nMatrix Multiplication:")
print(multiplication)
```

4. Using np.linspace() and np.zeros() and np.ones(), create three different arrays and stack them vertically using np.vstack(). Display the final stacked array.

Ans.

```
# Import NumPy library
import numpy as np

# Create an array using linspace()
array1 = np.linspace(1, 10, 5)

# Create an array of zeros
array2 = np.zeros(5)

# Create an array of ones
array3 = np.ones(5)

# Display individual arrays
print("Array using linspace():")
print(array1)

print("\nArray using zeros():")
```

```
print(array2)
```

```
print("\nArray using ones():")
```

```
print(array3)
```

```
# Stack arrays vertically
```

```
final_array = np.vstack((array1, array2, array3))
```

```
# Display final stacked array
```

```
print("\nFinal Stacked Array:")
```

```
print(final_array)
```

5. Create a DataFrame of at least 10 students with columns: Name, Age, Marks, City. Perform the following:

- Display the first 5 rows using head()
- Show basic statistics using describe()
- Filter students who scored more than 75 marks
- Sort by Marks in descending order

Ans.

```
# Import pandas library
```

```
import pandas as pd
```

```
# Create DataFrame of 10 students
```

```
data = {
```

```
    "Name": ["Harsh", "Aman", "Rahul", "Monu", "Priya",
```

```
"Riya","Rohan","Neha","Vikas","Karan"],  
  
"Age": [20,21,19,22,20,21,18,22,20,19],  
  
"Marks": [85,72,90,65,78,88,70,95,80,60],  
  
"City": ["Jaipur","Delhi","Mumbai","Agra","Kota",  
         "Noida","Pune","Ajmer","Jaipur","Delhi"]  
}
```

```
# Create DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display complete DataFrame
```

```
print("Student Data:")
```

```
print(df)
```

```
# Display first 5 rows
```

```
print("\nFirst 5 Rows:")
```

```
print(df.head())
```

```
# Show basic statistics
```

```
print("\nBasic Statistics:")
```

```
print(df.describe())
```

```
# Filter students with marks greater than 75
```

```
print("\nStudents scoring more than 75 marks:")
```

```
print(df[df["Marks"] > 75])
```

```
# Sort students by marks in descending order

print("\nStudents sorted by marks (Descending Order):")

print(df.sort_values(by="Marks", ascending=False))
```

6. Create a DataFrame with some missing values intentionally placed in the Marks and City columns. Then:

- Identify missing values using `isnull()`
- Fill missing marks with the column mean using `fillna()`
- Drop rows where City is missing using `dropna()`

Ans.

```
# Import pandas library

import pandas as pd

import numpy as np

# Create DataFrame with missing values

data = {

    "Name": ["Harsh", "Aman", "Rahul", "Monu", "Priya"],

    "Marks": [85, np.nan, 78, np.nan, 92],

    "City": ["Jaipur", "Delhi", np.nan, "Agra", np.nan]

}

# Create DataFrame
```



```
df = pd.DataFrame(data)

# Display original DataFrame
print("Original DataFrame:")
print(df)

# Identify missing values
print("\nMissing Values:")
print(df.isnull())

# Fill missing marks with mean value
mean_marks = df["Marks"].mean()

df["Marks"] = df["Marks"].fillna(mean_marks)

print("\nDataFrame after filling missing marks:")
print(df)

# Drop rows where City value is missing
new_df = df.dropna(subset=["City"])

print("\nDataFrame after removing missing City values:")
print(new_df)
```

7. Read a CSV file (you may create it manually) containing product sales data with columns: Product, Category, Sales, Region. Perform:

- Group by Category and calculate total Sales using `groupby()` and `agg()`
- Find the top 3 products by sales using `sort_values()` and `head()`

Ans.

```
# Import pandas library
```

```
import pandas as pd
```

```
# Create product sales data
```

```
data = {
```

```
    "Product": ["Laptop", "Mobile", "Tablet", "Headphone", "SmartWatch"],
```

```
    "Category": ["Electronics", "Electronics",
```

```
                "Electronics", "Accessories",
```

```
                "Accessories"],
```

```
    "Sales": [50000, 40000, 25000, 15000, 30000],
```

```
    "Region": ["North", "South", "East", "West", "North"]
```

```
}
```

```
# Convert into DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Save data into CSV file
```

```
df.to_csv("sales_data.csv", index=False)
```

```
# Read CSV file
```

```
sales_df = pd.read_csv("sales_data.csv")
```

```
# Display CSV data
```

```

print("Sales Data:")
print(sales_df)

# Group by Category and calculate total sales
print("\nTotal Sales by Category:")

category_sales = sales_df.groupby("Category").agg({
    "Sales": "sum"
})

print(category_sales)

# Find top 3 products by sales
print("\nTop 3 Products by Sales:")

top_products = sales_df.sort_values(
    by="Sales",
    ascending=False
).head(3)

print(top_products)

```

8. Merge two DataFrames:

- DataFrame 1: StudentID, Name, Department
- DataFrame 2: StudentID, Marks, Grade
- Merge on StudentID and display students who scored Grade 'A'

Ans.

```
# Import pandas library

import pandas as pd

# Create first DataFrame

data1 = {

    "StudentID": [101,102,103,104,105],

    "Name": ["Harsh","Aman","Rahul","Monu","Priya"],

    "Department": ["CSE","IT","AI","DS","CSE"]

}

df1 = pd.DataFrame(data1)

# Create second DataFrame

data2 = {

    "StudentID": [101,102,103,104,105],

    "Marks": [92,75,88,65,95],

    "Grade": ["A","B","A","C","A"]

}

df2 = pd.DataFrame(data2)

# Display first DataFrame
```

```
print("DataFrame 1:")
print(df1)

# Display second DataFrame
print("\nDataFrame 2:")
print(df2)

# Merge both DataFrames using StudentID
merged_df = pd.merge(df1, df2, on="StudentID")

print("\nMerged DataFrame:")
print(merged_df)

# Display students with Grade A
grade_a = merged_df[merged_df["Grade"]=="A"]

print("\nStudents with Grade A:")
print(grade_a)
```

9. Using Matplotlib, create the following plots from a student marks dataset (create the dataset yourself):

- A line plot showing marks trend across 5 subjects
- A bar chart comparing marks of 5 students
- Add proper title, xlabel, ylabel, legend, and grid to both plots

Ans.

```
# Import required libraries

import pandas as pd

import matplotlib.pyplot as plt
```

```
# Create dataset for student marks

subjects = ["Math", "Science", "English", "Python", "AI"]

marks = [85, 78, 90, 88, 95]

students = ["Harsh", "Aman", "Rahul", "Monu", "Priya"]

student_marks = [82, 75, 91, 70, 88]
```

```
# Create Line Plot

plt.figure(figsize=(8,5))

plt.plot(subjects,
         marks,
         marker='o',
         label="Marks Trend")

plt.title("Marks Trend Across Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.legend()
plt.grid()

plt.show()
```

```
# Create Bar Chart

plt.figure(figsize=(8,5))
```

```
plt.bar(students,
        student_marks,
        label="Student Marks")

plt.title("Comparison of Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.legend()
plt.grid()

plt.show()
```

10) Using Pandas built-in plotting, load or create a dataset with at least 4 numeric columns.

Create:

- A histogram for marks distribution
- A box plot to visualize spread and outliers
- A pie chart showing category-wise percentage

Ans.

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset with 4 numeric columns
data = {

    "Math": [78,85,90,67,88,72,95,80],
```

```
"Science":[82,79,91,70,84,75,93,85],  
"English":[75,88,92,69,86,78,90,82],  
"Computer":[90,95,89,72,87,80,96,84]  
}
```

```
df = pd.DataFrame(data)
```

```
# Histogram for marks distribution
```

```
df["Math"].plot(  
    kind="hist",  
    bins=5  
)
```

```
plt.title("Marks Distribution")
```

```
plt.xlabel("Marks")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

```
# Box Plot for spread and outliers
```

```
df.plot(kind="box")
```

```
plt.title("Box Plot of Student Marks")
```

```
plt.show()
```



```
# Pie Chart for category-wise percentage
```

```
category = {
```

```
    "A Grade":35,
```

```
    "B Grade":40,
```

```
    "C Grade":25
```

```
}
```

```
grade_data = pd.Series(category)
```

```
grade_data.plot(
```

```
    kind="pie",
```

```
    autopct="%1.1f%%"
```

```
)
```

```
plt.title("Category-wise Percentage")
```

```
plt.ylabel("")
```

```
plt.show()
```

11)Using Seaborn, create a dataset of at least 50 records with columns Age, Salary, Department, Experience. Plot:

- A scatter plot of Age vs Salary with hue as Department
- A heatmap showing correlation between numeric columns
- A bar plot showing average Salary per Department

Ans.

```
# Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create dataset with 50 records

data = {

    "Age": np.random.randint(22,50,50),

    "Salary": np.random.randint(30000,100000,50),

    "Department": np.random.choice(
        ["IT","HR","Sales","Marketing"],
        50
    ),

    "Experience": np.random.randint(1,15,50)
}

# Create DataFrame

df = pd.DataFrame(data)

# Display first 5 records

print("Dataset:")

print(df.head())
```

```
# Scatter Plot: Age vs Salary
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(
```

```
    data=df,
```

```
    x="Age",
```

```
    y="Salary",
```

```
    hue="Department"
```

```
)
```

```
plt.title("Age vs Salary")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Salary")
```

```
plt.grid()
```

```
plt.show()
```

```
# Heatmap for correlation
```

```
plt.figure(figsize=(8,5))
```

```
sns.heatmap(
```

```
    df.corr(numeric_only=True),
```

```
    annot=True
```

```
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

```
# Bar Plot for average salary by department
```

```
plt.figure(figsize=(8,5))
```

```
sns.barplot(
```

```
    data=df,
```

```
    x="Department",
```

```
    y="Salary"
```

```
)
```

```
plt.title("Average Salary per Department")
```

```
plt.xlabel("Department")
```

```
plt.ylabel("Average Salary")
```

```
plt.grid()
```

```
plt.show()
```

12) Create a subplot layout of 2 rows × 2 columns using Matplotlib containing:

- Line plot

- Scatter plot

- Bar chart

- Histogram All using the same dataset.

Add a main title using `suptitle()`

Ans.

```
# Import required libraries

import pandas as pd

import matplotlib.pyplot as plt


# Create dataset

data = {

    "Students": ["Harsh", "Aman", "Rahul", "Monu", "Priya"],

    "Marks": [85, 72, 90, 65, 78]

}


# Convert into DataFrame

df = pd.DataFrame(data)


# Create subplot layout (2 rows × 2 columns)

fig, ax = plt.subplots(2, 2, figsize=(10, 8))


# Line Plot

ax[0, 0].plot(

    df["Students"],

    df["Marks"]

)

ax[0, 0].set_title("Line Plot")


# Scatter Plot
```

```
ax[0,1].scatter(  
    df["Students"],  
    df["Marks"]  
)  
ax[0,1].set_title("Scatter Plot")
```

Bar Chart

```
ax[1,0].bar(  
    df["Students"],  
    df["Marks"]  
)  
ax[1,0].set_title("Bar Chart")
```

Histogram

```
ax[1,1].hist(  
    df["Marks"]  
)  
ax[1,1].set_title("Histogram")
```

Add main title

```
plt.suptitle("2 × 2 Subplots Using Same Dataset")
```

Adjust spacing

```
plt.tight_layout()
```

```
# Display plots
```

```
plt.show()
```